Lecture 3: signalling

thread factory, executor, future, thread pooling, backoff policy, signalling, condition variable, lost signal, spurious wakeup, wait set admission policy, blocking queue, predicate invalidation, thread-starvation deadlock, resource deadlock

Alexander Filatov
filatovaur@gmail.com

https://github.com/Svazars/parallel-programming/blob/main/slides/pdf/l3.pdf

In previous episodes

- Every thread has its own speed and scenario of execution
- We expect OS could pre-empt any thread at any time
- Interleaving model thread-safety: all operations with data structure
  - are non-overlapping intervals on single timeline
  - are consistent with each other in single-threaded sense
- **Mutual exclusion**: not more than one thread in critical section
- **Deadlock-freedom**: some thread eventually enters critical section
- **Starvation-freedom**: every thread eventually enters critical section
- Mutex design space: reentrancy, admission policy, fairness
- Mutex patterns: code locking, data locking, lock splitting
- Mutex best practices: recursive locks, encapsulation, leaf locking
- Documentation is you best friend

Lecture plan

1. Decoupling tasks from threads
   - Executor
   - ExecutorService
   - Thread pooling
2. Backoff policies
3. Signalling
4. Condition variable
   - API
   - Lost signal
   - Spurious wakeup
   - Fairness
   - Multiple condition variables per lock
   - Predicate invalidation
5. Thread pools
6. Summary

Useful interfaces

Runnable[1]

```
interface Runnable {
    void run();
}
```

- separate task (**what** to do) from environment (**where** to do)

[1] https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Runnable.html

Question time

Question: `Runnable.run` should be thread-safe?

Useful interfaces

```
interface Runnable { void run(); }
```

- separate task (**what** to do) from environment (**where** to do)

Useful interfaces

```
interface Runnable { void run(); }
```

- separate task (**what** to do) from environment (**where** to do)

ThreadFactory[2]

```
interface ThreadFactory {
  Thread newThread(Runnable r);
}
```

- separate environment (**what** available) from particular thread (**who** will do)

[2] https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/ThreadFactory.html

Question time

Question: `ThreadFactory.newThread` that returns "previously used" `Thread` instance: what are pros and cons?

Useful interfaces

```
interface Runnable { void run(); }
```

- separate task (**what** to do) from environment (**where** to do)

```
interface ThreadFactory { Thread newThread(Runnable r); }
```

- separate environment (**what** available) from particular thread (**who** will do)

Useful interfaces

```
interface Runnable { void run(); }
```

- separate task (**what** to do) from environment (**where** to do)

```
interface ThreadFactory { Thread newThread(Runnable r); }
```

- separate environment (**what** available) from particular thread (**who** will do)

Executor[3]

```
interface Executor {
  void execute(Runnable command);
}
```

- separate task submission (**how** distribute work) from the mechanics of how each task will be run (**who** will do, **when** will do)

[3] https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/Executor.html

Question time

Question: `Executor.execute` that performs task in the caller thread is OK?

Using executor

```
Executor executor = anExecutor();
executor.execute(new RunnableTask1());
executor.execute(new RunnableTask2());
```

Question time

Question: Do we need to use locks when using properly implemented thread-safe `Executor`?

Using executor

```
static long total = 0;
static Lock lock = new ReentrantLock();
static long compute(long input) { ... }
static void foo() {
  Executor executor = anExecutor();
  executor.execute( () -> {
        long tmp = compute(1);
        lock.lock(); try { total += tmp; } finally { lock.unlock(); }
  });
  executor.execute( () -> {
        long tmp = compute(2);
        lock.lock(); try { total += tmp; } finally { lock.unlock(); }
  });
```

ThreadPerTaskExecutor

```
class ThreadPerTaskExecutor implements Executor {
  private final ThreadFactory factory;
  public ThreadPerTaskExecutor(ThreadFactory f) {
    factory = f;
  }
  public void execute(Runnable r) {
    factory.newThread(r).start();
  }
}
```

Question time

Question: `Executor.execute` declares "fire-and-forget" API. Is it enough?

Lecture plan

1. Decoupling tasks from threads
   - Executor
   - ExecutorService
   - Thread pooling
2. Backoff policies
3. Signalling
4. Condition variable
   - API
   - Lost signal
   - Spurious wakeup
   - Fairness
   - Multiple condition variables per lock
   - Predicate invalidation
5. Thread pools
6. Summary

Even more useful interfaces

Callable<V>[4]

```
interface Callable<V> {
  V call() throws Exception;
}
```

- computation has 2 possible outcomes: value `V` or exception

[4] https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/Callable.html

Question time

Question: some thread experienced uncaught exception. Should we `exit` our program?

Even more useful interfaces

```
interface Callable<V> { V call() throws Exception; }
```

- computation has 2 possible outcomes: value `V` or exception

Even more useful interfaces

```
interface Callable<V> { V call() throws Exception; }
```

- computation has 2 possible outcomes: value V or exception

Future<V>[5]

```
interface Future<V> {
  V get();
  boolean isDone();
}
```

- represents the result of an asynchronous computation

[5] https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/Future.html

Question time

Question: what should `Future.get` return for `Callable<V>` that failed with exception?

Even more useful interfaces

```
interface Callable<V> { V call() throws Exception; }
```

- computation has 2 possible outcomes: value `V` or exception

```
interface Future<V> {
    V get();
    boolean isDone();
}
```

- represents the result of an asynchronous computation
- throws `ExecutionException` if the computation threw an exception

Even more useful interfaces

```
interface Callable<V> { V call() throws Exception; }
```

- computation has 2 possible outcomes: value `V` or exception

```
interface Future<V> {
    V get();
    boolean isDone();
}
```

- represents the result of an asynchronous computation
- throws `ExecutionException` if the computation threw an exception

Reminder: `Thread.join` does not propagate exceptions (see Task 1.1)

ExecutorService: beginning

ExecutorService[6]

```
interface ExecutorService extends Executor {
  void execute(Runnable command);
  <T> Future<T> submit(Callable<T> task);
  <T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks);
}
```

- features asynchronous computation
- allows bulk submission of tasks

[6] https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/ExecutorService.html

Reminder: ThreadPerTaskExecutor

```
class ThreadPerTaskExecutor implements Executor {
  private final ThreadFactory factory;
  public ThreadPerTaskExecutor(ThreadFactory f) { factory = f; }
  public void execute(Runnable r) { factory.newThread(r).start(); }
}
```

Reminder: ThreadPerTaskExecutor

```
class ThreadPerTaskExecutor implements Executor {
  private final ThreadFactory factory;
  public ThreadPerTaskExecutor(ThreadFactory f) { factory = f; }
  public void execute(Runnable r) { factory.newThread(r).start(); }
}
```

`ThreadPerTaskExecutor`**Service**:

- features asynchronous computation (`submit` returns `Future`)
- allows bulk submission of tasks (`submit(Collection)`)

ThreadPerTaskExecutorService

```
class ThreadPerTaskExecutorService implements ExecutorService {
  private final ThreadFactory factory;
  public ThreadPerTaskExecutorService(ThreadFactory f) { factory = f; }
  public void execute(Runnable r) {
    ???
  }
  <T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks) {
    ???
    ???
    ???
  }
  <T> Future<T> submit(Callable<T> task) {
    ???
  }}
```

ThreadPerTaskExecutorService

```
class ThreadPerTaskExecutorService implements ExecutorService {
  private final ThreadFactory factory;
  public ThreadPerTaskExecutorService(ThreadFactory f) { factory = f; }
  public void execute(Runnable r) {
    this.submit(() -> { r.run(); return null });
  }
  <T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks) {
    ???
    ???
    ???
  }
  <T> Future<T> submit(Callable<T> task) {
    ???
  }}
```

ThreadPerTaskExecutorService

```
class ThreadPerTaskExecutorService implements ExecutorService {
  private final ThreadFactory factory;
  public ThreadPerTaskExecutorService(ThreadFactory f) { factory = f; }
  public void execute(Runnable r) {
    this.submit(() -> { r.run(); return null });
  }
  <T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks) {
    var result = new ArrayList<Future<T>>();
    for (var t : tasks) { result.add(this.submit(t)); }
    return result;
  }
  <T> Future<T> submit(Callable<T> task) {
    ???
  }}
```

ThreadPerTaskExecutorService + JoinFuture

```
class ThreadPerTaskExecutorService implements ExecutorService {
  <T> Future<T> submit(Callable<T> task) {
    ???
    ???
    ???
    ???
  }
}
```

ThreadPerTaskExecutorService + JoinFuture

```
class ThreadPerTaskExecutorService implements ExecutorService {
  <T> Future<T> submit(Callable<T> task) {
    var future = new JoinFuture();
    future.thread = factory.newThread(() -> { future.result = task.call(); });
    future.thread.start();
    return future;
  }
}
```

ThreadPerTaskExecutorService + JoinFuture

```
class ThreadPerTaskExecutorService implements ExecutorService {
  <T> Future<T> submit(Callable<T> task) {
    var future = new JoinFuture();
    future.thread = factory.newThread(() -> { future.result = task.call(); });
    future.thread.start();
    return future;
  }
}
class JoinFuture<V> implements Future<V> {
  Thread thread;
  V result;
  public V get() { ??? }
  public boolean isDone() { ??? }
}
```

ThreadPerTaskExecutorService + JoinFuture

```
class ThreadPerTaskExecutorService implements ExecutorService {
  <T> Future<T> submit(Callable<T> task) {
    var future = new JoinFuture();
    future.thread = factory.newThread(() -> { future.result = task.call(); });
    future.thread.start();
    return future;
  }
}
class JoinFuture<V> implements Future<V> {
  Thread thread;
  V result;
  public V get() { thread.join(); return result; }
  public boolean isDone() { return !thread.isAlive(); }
}
```

Question time

Question: there are no locks to guard `JoinFuture.thread` and `JoinFuture.result`. Is it OK?

ThreadPerTaskExecutorService + JoinFuture = Homework

https://github.com/Svazars/parallel-programming/blob/main/hw/block1/3.1/readme.markdown

Homework

Task 3.1

- *Implement* `ThreadPerTaskExecutorService`
- *Implement* `JoinFuture` *that properly supports exceptions*

Question time

Question: what are negative sides for starting brand new thread for every submitted task?

Lecture plan

1. Decoupling tasks from threads
   - Executor
   - ExecutorService
   - Thread pooling
2. Backoff policies
3. Signalling
4. Condition variable
   - API
   - Lost signal
   - Spurious wakeup
   - Fairness
   - Multiple condition variables per lock
   - Predicate invalidation
5. Thread pools
6. Summary

Thread pooling

- `Thread` is valuable and scarce OS resource.
  - Caching could help.
- Too many `Threads` may slow-down each other (oversubscription).
  - Soft and hard limits could help.
- Most of the computations do not care which thread is used.
  - Decoupling tasks from carriers could help.
- `Threads` may have purpose and context (name, priority, task-specific limits).
  - Should be properly reflected in observability API.

SingleThreadExecutor + LockedFuture

Idea:

SingleThreadExecutor + LockedFuture

Idea:

- Only one thread to execute all tasks

SingleThreadExecutor + LockedFuture

Idea:

- Thread-safe FIFO container for submitted tasks
- Only one thread to execute all tasks

SingleThreadExecutor + LockedFuture

Idea:

- Lock-per-Future for `get`/`set` synchronization
- Thread-safe FIFO container for submitted tasks
- Only one thread to execute all tasks

SingleThreadExecutor + LockedFuture

- Lock-per-Future for `get`/`set` synchronization

SingleThreadExecutor + LockedFuture

- Lock-per-Future for get/set synchronization

```
class LockedFuture<V> implements Future<V> {
  final Lock l = new ReentrantLock(); final Callable<V> task;
  V result; boolean done = false;
  public LockedFuture(Callable<V> task) { this.task = task; }
  void setResult(V v) {
    l.lock(); try { result = v; done = true; } finally { l.unlock(); }}
  V get() {
    while (!isDone()) { /* await task completion */ }
    l.lock(); try { return result; } finally { l.unlock(); }
  }
  boolean isDone() { l.lock(); try { return done; } finally { l.unlock(); }}
}
```

SingleThreadExecutor + LockedFuture

- Lock-per-Future for `get`/`set` synchronization
- Thread-safe FIFO container

SingleThreadExecutor + LockedFuture

- Lock-per-Future for get/set synchronization
- Thread-safe FIFO container

```
class LockedFIFO<T> implements Queue<T> {
  private final Queue<T> q = new LinkedList<>();
  private final Lock l = new ReentrantLock();
  boolean offer(T e){ l.lock(); try { return q.offer(e); } finally { l.unlock()
  T poll()           { l.lock(); try { return q.poll();   } finally { l.unlock()
  boolean isEmpty() { l.lock(); try { return q.isEmpty();} finally { l.unlock()
  ...
}
```

SingleThreadExecutor + LockedFuture

- Lock-per-Future for `get`/`set` synchronization
- Thread-safe FIFO container for submitted tasks

SingleThreadExecutor + LockedFuture

- Lock-per-Future for get/set synchronization
- Thread-safe FIFO container for submitted tasks

```
class SingleThreadExecutor implements ExecutorService {
  private final ThreadFactory factory;
  private final Queue<LockedFuture<?>> queue = new LockedFIFO<>();
  public SingleThreadExecutor(ThreadFactory f) { factory = f; }
  <T> Future<T> submit(Callable<T> task) {
    var future = new LockedFuture<>(task);
    queue.offer(future);
    return future;
  }
}
```

SingleThreadExecutor + LockedFuture

- Lock-per-Future for `get`/`set` synchronization
- Thread-safe FIFO container for submitted tasks
- Only one thread to execute all tasks

SingleThreadExecutor + LockedFuture

- Lock-per-Future for get/set synchronization
- Thread-safe FIFO container for submitted tasks
- Only one thread to execute all tasks

```
class SingleThreadExecutor implements ExecutorService {
  private final ThreadFactory factory;
  private final Queue<LockedFuture<?>> queue = new LockedFIFO<>();
  private final Thread worker = factory.newThread(() -> {
    while (true) {
      while (queue.isEmpty()) { /* await new task */ }
      var f = queue.poll();
      f.setResult(f.task.call());
    }});
}
```

Question time

Question:

```
LockedFuture.get: while (!isDone()) { /* await task completion */ }
SingleThreadExecutor: while (queue.isEmpty()) { /* await new task */ }
```

Could you spot **performance** problem here?

Lecture plan

1. Decoupling tasks from threads
   - Executor
   - ExecutorService
   - Thread pooling
2. Backoff policies
3. Signalling
4. Condition variable
   - API
   - Lost signal
   - Spurious wakeup
   - Fairness
   - Multiple condition variables per lock
   - Predicate invalidation
5. Thread pools
6. Summary

NoBackoff policy

```
while (true) {
    if (isDone()) {
        break;
    }
    // immediately try again
}
```

NoBackoff policy

```
while (true) {
    if (isDone()) {
        break;
    }
    // immediately try again
}
```

- Pro: highest possible response time. Could be blazingly fast, especially when polling thread does not context switch

NoBackoff policy

```
while (true) {
    if (isDone()) {
        break;
    }
    // immediately try again
}
```

- Pro: highest possible response time. Could be blazingly fast, especially when polling thread does not context switch
- Con: could waste CPU time

NoBackoff policy

```
while (true) {
    if (isDone()) {
        break;
    }
    // immediately try again
}
```

- Pro: highest possible response time. Could be blazingly fast, especially when polling thread does not context switch
- Con: could waste CPU time
- Con: could cause starvation

Question time

Question: What should thread do when task is not yet ready?

FixedBackOff policy

```
while (true) {
    if (isDone()) {
        break;
    }
    Thread.sleep(CONSTANT);
}
```

FixedBackOff policy

```
while (true) {
    if (isDone()) {
        break;
    }
    Thread.sleep(CONSTANT);
}
```

- Pro: easy to implement

FixedBackOff policy

```
while (true) {
    if (isDone()) {
        break;
    }
    Thread.sleep(CONSTANT);
}
```

- Pro: easy to implement
- Pro: predictable

FixedBackOff policy

```
while (true) {
    if (isDone()) {
        break;
    }
    Thread.sleep(CONSTANT);
}
```

- Pro: easy to implement
- Pro: predictable
- Pro: saves CPU resources

FixedBackOff policy

```
while (true) {
    if (isDone()) {
        break;
    }
    Thread.sleep(CONSTANT);
}
```

- Pro: easy to implement
- Pro: predictable
- Pro: saves CPU resources
- Con: optimal `CONSTANT` could be task-specific, CPU-specific, workload-specific

FixedBackOff policy

```
while (true) {
    if (isDone()) {
        break;
    }
    Thread.sleep(CONSTANT);
}
```

- Pro: easy to implement
- Pro: predictable
- Pro: saves CPU resources
- Con: optimal `CONSTANT` could be task-specific, CPU-specific, workload-specific
  - could cause starvation (too small delay)

FixedBackOff policy

```
while (true) {
    if (isDone()) {
        break;
    }
    Thread.sleep(CONSTANT);
}
```

- Pro: easy to implement
- Pro: predictable
- Pro: saves CPU resources
- Con: optimal `CONSTANT` could be task-specific, CPU-specific, workload-specific
  - could cause starvation (too small delay)
  - could cause latency problems (adds delay to observed pauses)

FixedBackOff policy

```
while (true) {
    if (isDone()) {
        break;
    }
    Thread.sleep(CONSTANT);
}
```

- Pro: easy to implement
- Pro: predictable
- Pro: saves CPU resources
- Con: optimal `CONSTANT` could be task-specific, CPU-specific, workload-specific
  - could cause starvation (too small delay)
  - could cause latency problems (adds delay to observed pauses)
  - could cause throughput degradation (too big delay)

ExponentialBackOff policy

```
long current_delay = START_DELAY;
while (true) {
    if (isDone()) {
        break;
    }
    sleep(current_delay);
    current_delay = min(current_delay * 2, MAX_DELAY);
}
```

ExponentialBackOff policy

```
long current_delay = START_DELAY;
while (true) {
    if (isDone()) {
        break;
    }
    sleep(current_delay);
    current_delay = min(current_delay * 2, MAX_DELAY);
}
```

- Pro: saves CPU resources

ExponentialBackOff policy

```
long current_delay = START_DELAY;
while (true) {
    if (isDone()) {
        break;
    }
    sleep(current_delay);
    current_delay = min(current_delay * 2, MAX_DELAY);
}
```

- Pro: saves CPU resources
- Pro: latency-friendly

ExponentialBackOff policy

```
long current_delay = START_DELAY;
while (true) {
    if (isDone()) {
        break;
    }
    sleep(current_delay);
    current_delay = min(current_delay * 2, MAX_DELAY);
}
```

- Pro: saves CPU resources
- Pro: latency-friendly
- Con: still not perfect responsiveness

ExponentialBackOff policy

```
long current_delay = START_DELAY;
while (true) {
    if (isDone()) {
        break;
    }
    sleep(current_delay);
    current_delay = min(current_delay * 2, MAX_DELAY);
}
```

- Pro: saves CPU resources
- Pro: latency-friendly
- Con: still not perfect responsiveness

We need something better than polling (sleep-and-retry)!

Lecture plan

1. Decoupling tasks from threads
   - Executor
   - ExecutorService
   - Thread pooling
2. Backoff policies
3. Signalling
4. Condition variable
   - API
   - Lost signal
   - Spurious wakeup
   - Fairness
   - Multiple condition variables per lock
   - Predicate invalidation
5. Thread pools
6. Summary

Signalling

Motivation

Thread offloads tasks to other threads and waits for completion.
Polling with backoff: "time-to-react" (latency) vs. "unproductive time" (throughput).

Signalling

Motivation

Thread offloads tasks to other threads and waits for completion.
Polling with backoff: "time-to-react" (latency) vs. "unproductive time" (throughput).
We need to

- subscribe on completion event
- notify subscribers (other threads) in efficient way

We need some support from OS!

Signalling

Idea

Blocking API:

- `wait` (a.k.a. `await`, `subscribe` ...)

Signalling

Idea

Blocking API:

- `wait` (a.k.a. `await`, `subscribe` ...)

What are we waiting for?

- Some task to be completed (execution flow control)
- Some data to be available (data flow control)

Signalling

Idea

Blocking API:

- `wait` (a.k.a. `await`, `subscribe` ...)

What are we waiting for?

- Some task to be completed (execution flow control)
- Some data to be available (data flow control)

Non-blocking API:

- `notify` (`notify_one`, `signal` ...)
- `notifyAll` (`signalAll`, `broadcast` ...)

Signalling

Idea

Blocking API:

- `wait` (a.k.a. `await`, `subscribe` ...)

What are we waiting for?

- Some task to be completed (execution flow control)
- Some data to be available (data flow control)

Non-blocking API:

- `notify` (`notify_one`, `signal` ...)
- `notifyAll` (`signalAll`, `broadcast` ...)

Who is going to be notified?

Signalling

Idea

Blocking API:

- `wait` (a.k.a. `await`, `subscribe` ...)

What are we waiting for?

- Some task to be completed (execution flow control)
- Some data to be available (data flow control)

Non-blocking API:

- `notify` (`notify_one`, `signal` ...)
- `notifyAll` (`signalAll`, `broadcast` ...)

Who is going to be notified?

- Only threads interested in communication
- The one/ones that are now suspended in `wait` method

Signalling

Idea

Blocking API:

- `wait` (a.k.a. `await`, `subscribe` ...)

What are we waiting for?

- Some task to be completed (execution flow control)
- Some data to be available (data flow control)

Non-blocking API:

- `notify` (`notify_one`, `signal` ...)
- `notifyAll` (`signalAll`, `broadcast` ...)

Who is going to be notified?

- Only threads interested in communication
- The one/ones that are now suspended in `wait` method

Must avoid race conditions and data races!

Question time

Question: concurrency primitive that you already know that helps to avoid data races?

Lecture plan

1. Decoupling tasks from threads
   - Executor
   - ExecutorService
   - Thread pooling
2. Backoff policies
3. Signalling
4. Condition variable
   - API
   - Lost signal
   - Spurious wakeup
   - Fairness
   - Multiple condition variables per lock
   - Predicate invalidation
5. Thread pools
6. Summary

Condition variable API

Creation[7]:

```
interface Lock {
    Condition newCondition();
}
```

Operations[8]:

```
interface Condition {
    void await();
    void signal();
    void signalAll();
}
```

[7] https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/locks/Lock.html#newCondition()
[8] https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/locks/Condition.html

Condition variable

Usage rules

Illegal operations:

- `await` out of critical section
- `signal` out of critical section

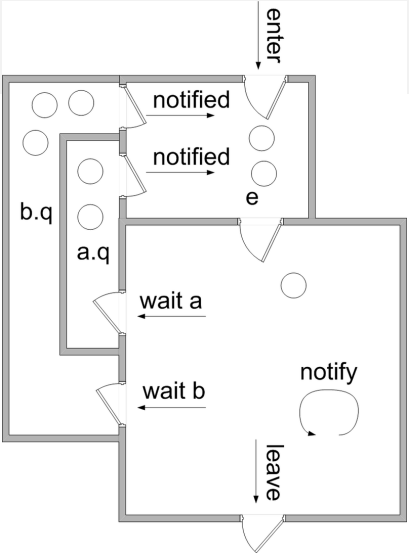
Important details:

- `await` unlocks mutex and starts listening for notification **atomically**
- `await` wake-ups (consumes notification) and **starts** mutex acquisition
- `signal/signalAll` wakes up waiting thread(s) only

Condition variable

Overview

One of possible high-level designs[9]:



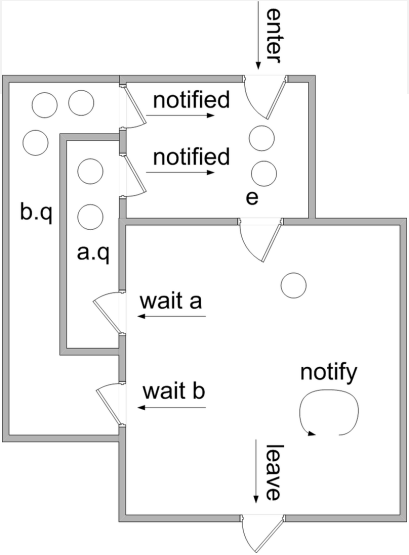
[9] https://en.wikipedia.org/wiki/Monitor_(synchronization)#/media/File:Monitor_(synchronization)-Mesa.png

Condition variable

Overview

One of possible high-level designs[9]:

- Mutex: Owner
  - The only thread inside critical section



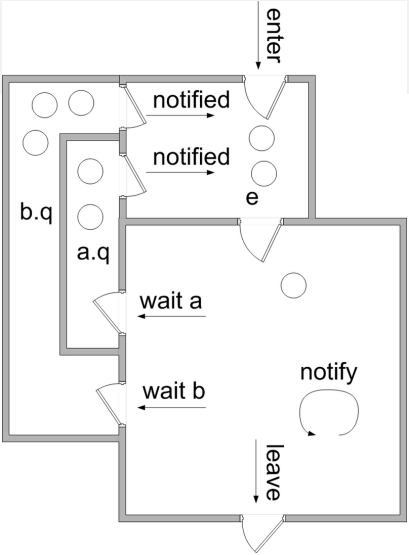
[9] https://en.wikipedia.org/wiki/Monitor_(synchronization)#/media/File:Monitor_(synchronization)-Mesa.png

Condition variable

Overview

One of possible high-level designs[9]:

- Mutex: Owner
  - The only thread inside critical section
- Mutex: EnterSet
  - Threads trying to enter critical section
  - Admission policy for lock acquisition



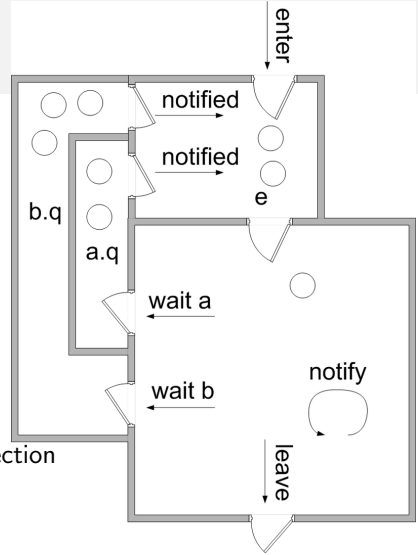
[9] https://en.wikipedia.org/wiki/Monitor_(synchronization)#/media/File:Monitor_(synchronization)-Mesa.png

Condition variable

Overview

One of possible high-level designs[9]:

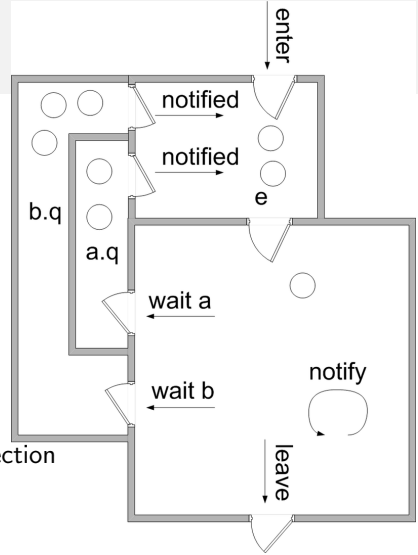
- Mutex: Owner
  - The only thread inside critical section
- Mutex: EnterSet
  - Threads trying to enter critical section
  - Admission policy for lock acquisition
- Condition: WaitSet
  - Threads waiting for signal and now out of critical section
  - Admission policy for notification



[9] https://en.wikipedia.org/wiki/Monitor_(synchronization)#/media/File:Monitor_(synchronization)-Mesa.png

Condition variable

Overview



One of possible high-level designs[9]:

- Mutex: Owner
  - The only thread inside critical section
- Mutex: EnterSet
  - Threads trying to enter critical section
  - Admission policy for lock acquisition
- Condition: WaitSet
  - Threads waiting for signal and now out of critical section
  - Admission policy for notification

Important: notified threads need to re-enter mutex

[9] https://en.wikipedia.org/wiki/Monitor_(synchronization)#/media/File:Monitor_(synchronization)-Mesa.png

Reminder: LockedFuture

```
class LockedFuture<V> implements Future<V> {
  final Lock l = new ReentrantLock();
  final Callable<V> task;
  V result;
  boolean done = false;
  public LockedFuture(Callable<V> task) { this.task = task; }
  ...
```

Reminder: LockedFuture

```
class LockedFuture<V> implements Future<V> { Lock l; V result; boolean done;
  void setResult(V v) {
    l.lock(); try { result = v; done = true; } finally { l.unlock(); }
  }
  V get() {
    while (!isDone()) { /* await task completion */ } // <-----------------
    l.lock(); try { return result; } finally { l.unlock(); }
  }
  boolean isDone() { l.lock(); try { return done; } finally { l.unlock(); }}
}
```

BetterFuture: using condition variables

```
class BetterFuture<V> implements Future<V> { Lock l; V result; boolean done;
  final Condition c = l.newCondition();
  V get() { ??? }



  void setResult(V v) { ??? }
```

BetterFuture: using condition variables

```
class BetterFuture<V> implements Future<V> { Lock l; V result; boolean done;
  final Condition c = l.newCondition();
  V get() { l.lock(); try {
              ???
              ???
            } finally { l.unlock(); }}
  void setResult(V v) {
            l.lock(); try {
              ???
              ???
            } finally { l.unlock(); }}}
```

BetterFuture: using condition variables

```
class BetterFuture<V> implements Future<V> { Lock l; V result; boolean done;
  final Condition c = l.newCondition();
  V get() { l.lock(); try {
              c.await();
              return result;
            } finally { l.unlock(); }}
  void setResult(V v) {
            l.lock(); try {
              result = v; done = true;
              c.signal();
            } finally { l.unlock(); }}}
```

BetterFuture: using condition variables

```
class BetterFuture<V> implements Future<V> { Lock l; V result; boolean done;
  final Condition c = l.newCondition();
  V get() { l.lock(); try {
              c.await();
              return result;
            } finally { l.unlock(); }}
  void setResult(V v) {
            l.lock(); try {
              result = v; done = true;
              c.signal();
            } finally { l.unlock(); }}}
```

Reminder: `signal` wakes up one of waiting threads

BetterFuture: using condition variables

```
class BetterFuture<V> implements Future<V> { Lock l; V result; boolean done;
  final Condition c = l.newCondition();
  V get() { l.lock(); try {
              c.await();
              return result;
            } finally { l.unlock(); }}
  void setResult(V v) {
            l.lock(); try {
              result = v; done = true;
              c.signal();
            } finally { l.unlock(); }}}
```

Reminder: `signal` wakes up one of waiting threads

Signal from `setResult` could be lost, `get` would block forever

Lecture plan

1. Decoupling tasks from threads
   - Executor
   - ExecutorService
   - Thread pooling
2. Backoff policies
3. Signalling
4. Condition variable
   - API
   - Lost signal
   - Spurious wakeup
   - Fairness
   - Multiple condition variables per lock
   - Predicate invalidation
5. Thread pools
6. Summary

Problem №1: lost signal due to race condition

Short description: `WaitSet` is empty due to race condition

```
threadA() {
    lock.lock();
    try {
        ready = true; condition.signal();
    } finally { lock.unlock(); }
}
threadB() {
    lock.lock();
    try {
        condition.await()
        if (ready) doComputation();
    } finally { lock.unlock(); }
}
```

Problem №1: lost signal due to race condition

Short description: `WaitSet` is empty due to race condition
Solution: never expect that your `signal` will reach already waiting receiver.
Sender thread:

- mark ready
- then `signal`

Receiver thread:

- check ready
- `await` if not yet ready

Reminder: BetterFuture

... that lose signals here and there

```
class BetterFuture<V> implements Future<V> { Lock l; V result; boolean done;
  final Condition c = l.newCondition();
  V get() { l.lock(); try {
              c.await();
              return result;
            } finally { l.unlock(); }}
  void setResult(V v) {
            l.lock(); try {
              result = v; done = true;
              c.signal();
            } finally { l.unlock(); }}}
```

EvenBetterFuture: using condition variables

... without lost signal due to race condition

```
class EvenBetterFuture<V> implements Future<V> { Lock l; V result; boolean done
  final Condition c = l.newCondition();
  V get() { l.lock(); try {
              if (!done) { c.await(); } // guard against too fast `setResult`
              return result;
            } finally { l.unlock(); }}
  void setResult(V v) {
            l.lock(); try {
              result = v; done = true;
              c.signal();
            } finally { l.unlock(); }}}
```

EvenBetterFuture: using condition variables

... without lost signal due to race condition

```
class EvenBetterFuture<V> implements Future<V> { Lock l; V result; boolean done
  final Condition c = l.newCondition();
  V get() { l.lock(); try {
              if (!done) { c.await(); } // guard against too fast `setResult`
              return result;
            } finally { l.unlock(); }}
  void setResult(V v) {
            l.lock(); try {
              result = v; done = true;
              c.signal();
            } finally { l.unlock(); }}}
```

Seems OK now...

EvenBetterFuture: using condition variables

... without lost signal due to race condition

```
class EvenBetterFuture<V> implements Future<V> { Lock l; V result; boolean done
  final Condition c = l.newCondition();
  V get() { l.lock(); try {
              if (!done) { c.await(); } // guard against too fast `setResult`
              return result;
            } finally { l.unlock(); }}
  void setResult(V v) {
            l.lock(); try {
              result = v; done = true;
              c.signal();
            } finally { l.unlock(); }}}
```

Seems OK now... What if there are 2 threads invoking get?

Problem №2: lost signal due to logic error

Short description: `signal` delivered message to wrong waiter

Problem №2: lost signal due to logic error

Short description: `signal` delivered message to wrong waiter
Solutions:

Problem №2: lost signal due to logic error

Short description: `signal` delivered message to wrong waiter
Solutions:

- `signalAll` is always safer, yet could be less performant. **Not always correct**.

Problem №2: lost signal due to logic error

Short description: `signal` delivered message to wrong waiter
Solutions:

- `signalAll` is always safer, yet could be less performant. **Not always correct**.
- better divide responsibility across threads (one-to-one communication via personal `Conditions`? other sync primitives?)

Homework: lost signal

https://github.com/Svazars/parallel-programming/blob/main/hw/block1/3.2/readme.markdown

Homework

Task 3.2 Find at least two bugs in sketch implementation of "two producer, single consumer" concurrent system that uses `java.util.concurrent.Condition`.

TheBestFuture: using condition variables

... without lost signal due to logic error

```
class TheBestFuture<V> implements Future<V> { Lock l; V result; boolean done;
  final Condition c = l.newCondition();
  V get() { l.lock(); try {
              if (!done) { c.await(); }
              return result;
            } finally { l.unlock(); }}
  void setResult(V v) {
            l.lock(); try {
              result = v; done = true;
              c.signalAll(); // notify all `get`-ers
            } finally { l.unlock(); }}}
```

TheBestFuture: using condition variables

... without lost signal due to logic error

```
class TheBestFuture<V> implements Future<V> { Lock l; V result; boolean done;
  final Condition c = l.newCondition();
  V get() { l.lock(); try {
              if (!done) { c.await(); }
              return result;
            } finally { l.unlock(); }}
  void setResult(V v) {
            l.lock(); try {
              result = v; done = true;
              c.signalAll(); // notify all `get`-ers
            } finally { l.unlock(); }}}
```

Seems OK now...

TheBestFuture: using condition variables

... without lost signal due to logic error

```
class TheBestFuture<V> implements Future<V> { Lock l; V result; boolean done;
  final Condition c = l.newCondition();
  V get() { l.lock(); try {
              if (!done) { c.await(); }
              return result;
            } finally { l.unlock(); }}
  void setResult(V v) {
            l.lock(); try {
              result = v; done = true;
              c.signalAll(); // notify all `get`-ers
            } finally { l.unlock(); }}}
```

Seems OK now... But I have an itch to re-read documentation for some reason

Lecture plan

1. Decoupling tasks from threads
   - Executor
   - ExecutorService
   - Thread pooling
2. Backoff policies
3. Signalling
4. Condition variable
   - API
   - Lost signal
   - Spurious wakeup
   - Fairness
   - Multiple condition variables per lock
   - Predicate invalidation
5. Thread pools
6. Summary

Problem №3: spurious wakeup

Short description: thread wake-ups from `condition.await` for no reason[10,11]

```
main() {
    lock.lock();
    try {
        condition.await();
        System.out.println("Should not reach here");
    } finally { lock.unlock(); }
}
```

[10] https://pubs.opengroup.org/onlinepubs/009604599/functions/pthread_cond_signal.html
[11] https://devblogs.microsoft.com/oldnewthing/20180201-00/?p=97946

Problem №3: spurious wakeup

Short description: thread wake-ups from `condition.await` for no reason[10,11]

```
main() {
    lock.lock();
    try {
        condition.await();
        System.out.println("Should not reach here");
    } finally { lock.unlock(); }
}
```

Solution:

- always wrap `await` into `while` loop with predicate re-check
- always expect that `await` is implemented as `unlock(); sleep(1); lock();`

[10] https://pubs.opengroup.org/onlinepubs/009604599/functions/pthread_cond_signal.html
[11] https://devblogs.microsoft.com/oldnewthing/20180201-00/?p=97946

CondVarFuture

```
class CondVarFuture<V> implements Future<V> { Lock l; V result; boolean done;
  final Condition c = l.newCondition();
  V get() { l.lock(); try {
              while (!done) { c.await(); } // guard against spurious wakeups
              return result;
            } finally { l.unlock(); }}
  void setResult(V v) {
            l.lock(); try {
              result = v; done = true;
              c.signalAll();
            } finally { l.unlock(); }}}
```

CondVarFuture

```
class CondVarFuture<V> implements Future<V> { Lock l; V result; boolean done;
  final Condition c = l.newCondition();
  V get() { l.lock(); try {
              while (!done) { c.await(); } // guard against spurious wakeups
              return result;
            } finally { l.unlock(); }}
  void setResult(V v) {
            l.lock(); try {
              result = v; done = true;
              c.signalAll();
            } finally { l.unlock(); }}}
```

Seems OK now...

CondVarFuture

```
class CondVarFuture<V> implements Future<V> { Lock l; V result; boolean done;
  final Condition c = l.newCondition();
  V get() { l.lock(); try {
              while (!done) { c.await(); } // guard against spurious wakeups
              return result;
            } finally { l.unlock(); }}
  void setResult(V v) {
            l.lock(); try {
              result = v; done = true;
              c.signalAll();
            } finally { l.unlock(); }}}
```

Seems OK now... Does not properly support exceptions, by the way

CondVarFuture

https://github.com/Svazars/parallel-programming/blob/main/hw/block1/3.3/readme.markdown

Homework

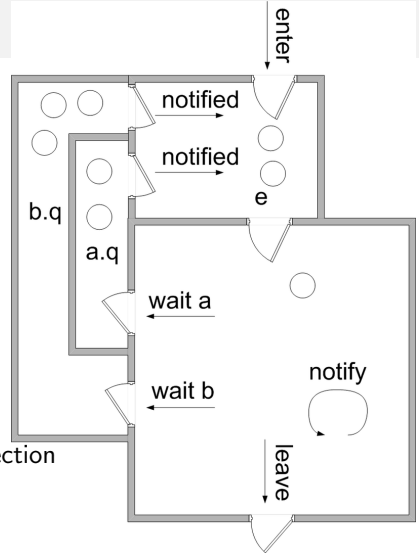
Task 3.3

- *Implement `SingleTheadExecutorService` that properly supports severe `Throwables` by starting new worker thread*
- *Implement `CondVarFuture` that properly supports exceptions using `java.util.concurrent.Condition`*

Lecture plan

1. Decoupling tasks from threads
   - Executor
   - ExecutorService
   - Thread pooling
2. Backoff policies
3. Signalling
4. Condition variable
   - API
   - Lost signal
   - Spurious wakeup
   - Fairness
   - Multiple condition variables per lock
   - Predicate invalidation
5. Thread pools
6. Summary

Reminder: condition variable overview



One of possible high-level designs:

- Mutex: Owner
  - The only thread inside critical section
- Mutex: EnterSet
  - Threads trying to enter critical section
  - **Admission policy for lock acquisition**
- Condition: WaitSet
  - Threads waiting for signal and now out of critical section
  - **Admission policy for notification**

Question time

Question: what is the connection between starvation and admission policy?

Question time

Question: what is the connection between fairness and throughput?

Fairness and Starvation for Condition Variables

Short description: fair mutex + unfair `WaitSet` => unfair system

```
threadA()  { l.lock(); try { while(true) {
                                System.out.println("A");
                                condition.signal(); condition.await();
}} finally { l.unlock(); }}
threadB()  { l.lock(); try { while(true) {
                                System.out.println("B");
                                condition.signal(); condition.await();
}} finally { l.unlock(); }}
threadC()  { l.lock(); try { while(true) {
                                System.out.println("C");
                                condition.signal(); condition.await();
}} finally { l.unlock(); }}
```

Fairness and Starvation for Condition Variables

Short description: fair mutex + unfair `WaitSet` => unfair system

```
threadA()  { l.lock(); try { while(true) {
                                System.out.println("A");
                                condition.signal(); condition.await();
}} finally { l.unlock(); }}
threadB()  { l.lock(); try { while(true) {
                                System.out.println("B");
                                condition.signal(); condition.await();
}} finally { l.unlock(); }}
threadC()  { l.lock(); try { while(true) {
                                System.out.println("C");
                                condition.signal(); condition.await();
}} finally { l.unlock(); }}
```

A, B, A, B, A, B ...

Unbounded concurrent queue

- Thread-safe FIFO container

```
class LockedFIFO<T> implements Queue<T> {
    private final Queue<T> q = new LinkedList<>();
    private final Lock l = new ReentrantLock();
    boolean offer(T e) { l.lock(); try { return q.offer(e); } finally { l.unloc
    T poll()           { l.lock(); try { return q.poll();   } finally { l.unloc
    boolean isEmpty()  { l.lock(); try { return q.isEmpty();} finally { l.unloc
}
```

Question time

Question: could you estimate the size of `LockedFIFO`?

Bounded concurrent queue

- FIFO container with fixed size

Question time

Question: what submitting thread should do when queue is full?

Question time

Question: what executor thread should do when queue is empty?

Bounded concurrent queue

- FIFO container with fixed size and blocking methods

Bounded concurrent queue

- FIFO container with fixed size and blocking methods

BlockingQueue<E>[12]

```
interface BlockingQueue<E> {
    void put(E e);
    E take();
}
```

[12] https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/BlockingQueue.html

Bounded concurrent queue

- FIFO container with fixed size and blocking methods

`BlockingQueue<E>`[12]

```
interface BlockingQueue<E> {
    void put(E e);
    E take();
}
```

Mmmm... `add`, `offer`, `put`, `remove`, `poll` ...

[12] https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/BlockingQueue.html

Bounded concurrent queue

- FIFO container with fixed size and blocking methods

```
interface BlockingQueue<E> {
    void put(E e);
    E take();
}
```

Bounded concurrent queue

- FIFO container with fixed size and blocking methods

```
interface BlockingQueue<E> {
    void put(E e);
    E take();
}
```

Refresher on Java stdlib collections[13]

	Throws exception	Special value	Blocks	Times out
Insert	add(e)	offer(e)	put(e)	offer(e, time, unit)
Remove	remove()	poll()	take()	poll(time, unit)
Examine	element()	peek()	N/A	N/A

[13] https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/BlockingQueue.html

Bounded concurrent queue with single condition

```
class BoundedBuffer<E> implements BlockingQueue<E> {
  Lock l = new ReentrantLock();
  Condition c = lock.newCondition();
  Queue<E> q = new LinkedList<>();
  ...
```

Bounded concurrent queue with single condition

```
class BoundedBuffer<E> implements BlockingQueue<E> { Lock l; Condition c; Queue
  ...
```

Bounded concurrent queue with single condition

```
class BoundedBuffer<E> implements BlockingQueue<E> { Lock l; Condition c; Queue
  void put(E x) { l.lock(); try {
      ???
      ???
    } finally { l.unlock(); }}
  E take() { lock.lock(); try {
      ???
      ???
      ???
    } finally { l.unlock(); }}}
```

Bounded concurrent queue with single condition

```
class BoundedBuffer<E> implements BlockingQueue<E> { Lock l; Condition c; Queue
  void put(E x) { l.lock(); try {
      while (q.size() == MAX_CAPACITY) { c.await(); }
      q.add(x); c.signal();
    } finally { l.unlock(); }}
  E take() { lock.lock(); try {
      ???
      ???
      ???
    } finally { l.unlock(); }}}
```

Bounded concurrent queue with single condition

```
class BoundedBuffer<E> implements BlockingQueue<E> { Lock l; Condition c; Queue
  void put(E x) { l.lock(); try {
      while (q.size() == MAX_CAPACITY) { c.await(); }
      q.add(x); c.signal();
    } finally { l.unlock(); }}
  E take() { lock.lock(); try {
      while (q.isEmpty()) { c.await(); }
      c.signal();
      return q.remove();
    } finally { l.unlock(); }}}
```

Bounded concurrent queue with single condition

```
class BoundedBuffer<E> implements BlockingQueue<E> { Lock l; Condition c; Queue
  void put(E x) { l.lock(); try {
      while (q.size() == MAX_CAPACITY) { c.await(); }
      q.add(x); c.signal();
    } finally { l.unlock(); }}
  E take() { lock.lock(); try {
      while (q.isEmpty()) { c.await(); }
      c.signal();
      return q.remove();
    } finally { l.unlock(); }}}
```

Reminder: `signal` wakes up one of waiting threads

Bounded concurrent queue with single condition

```
class BoundedBuffer<E> implements BlockingQueue<E> { Lock l; Condition c; Queue
  void put(E x) { l.lock(); try {
      while (q.size() == MAX_CAPACITY) { c.await(); }
      q.add(x); c.signal();
    } finally { l.unlock(); }}
  E take() { lock.lock(); try {
      while (q.isEmpty()) { c.await(); }
      c.signal();
      return q.remove();
    } finally { l.unlock(); }}}
```

Reminder: `signal` wakes up one of waiting threads

Assume: `MAX_CAPACITY = 2`, `q.size() == 2`, 3 blocked puts

Bounded concurrent queue with single condition

```
class BoundedBuffer<E> implements BlockingQueue<E> { Lock l; Condition c; Queue
  void put(E x) { l.lock(); try {
      while (q.size() == MAX_CAPACITY) { c.await(); }
      q.add(x); c.signal();
    } finally { l.unlock(); }}
  E take() { lock.lock(); try {
      while (q.isEmpty()) { c.await(); }
      c.signal();
      return q.remove();
    } finally { l.unlock(); }}}
```

Reminder: `signal` wakes up one of waiting threads

Assume: `MAX_CAPACITY = 2`, `q.size() == 2`, 3 blocked puts

Scenario: 3 takes, last still hangs

Homework: analysis of bounded concurrent queue with single condition

https://github.com/Svazars/parallel-programming/blob/main/hw/block1/3.4/readme.markdown

Homework

Task 3.4 Prove that for any `MAX_CAPACITY` > 2 there exists concurrent execution scenario where `take` operation indefinitely blocks despite the fact that queue contains some elements.

Lecture plan

1. Decoupling tasks from threads
   - Executor
   - ExecutorService
   - Thread pooling
2. Backoff policies
3. Signalling
4. Condition variable
   - API
   - Lost signal
   - Spurious wakeup
   - Fairness
   - Multiple condition variables per lock
   - Predicate invalidation
5. Thread pools
6. Summary

Bounded concurrent queue with two conditions

```
class BoundedBuffer<E> implements BlockingQueue<E> { Lock l; Queue<E> q;
  Condition notFull  = l.newCondition();
  Condition notEmpty = l.newCondition();
  void put(E x) {
      ???
      ???
      ???}
  E take() {
      ???
      ???
      ???
      ???}
```

Bounded concurrent queue with two conditions

```
class BoundedBuffer<E> implements BlockingQueue<E> { Lock l; Queue<E> q;
  Condition notFull  = l.newCondition();
  Condition notEmpty = l.newCondition();
  void put(E x) { l.lock(); try {
      ???
      ???
    } finally { l.unlock(); }}
  E take() { lock.lock(); try {
      ???
      ???
      ???
    } finally { l.unlock(); }}}
```

Bounded concurrent queue with two conditions

```
class BoundedBuffer<E> implements BlockingQueue<E> { Lock l; Queue<E> q;
  Condition notFull  = l.newCondition();
  Condition notEmpty = l.newCondition();
  void put(E x) { l.lock(); try {
      while (q.size() == MAX_CAPACITY) { notFull.await(); }
      q.add(x); notEmpty.signal();
    } finally { l.unlock(); }}
  E take() { lock.lock(); try {
      while (q.isEmpty()) { notEmpty.await(); }
      notFull.signal();
      return q.remove();
    } finally { l.unlock(); }}}
```

Bounded concurrent queue with two conditions

```
class BoundedBuffer<E> implements BlockingQueue<E> { Lock l; Queue<E> q;
  Condition notFull  = l.newCondition();
  Condition notEmpty = l.newCondition();
  void put(E x) { l.lock(); try {
      while (q.size() == MAX_CAPACITY) { notFull.await(); }
      q.add(x); notEmpty.signal();
    } finally { l.unlock(); }}
  E take() { lock.lock(); try {
      while (q.isEmpty()) { notEmpty.await(); }
      notFull.signal();
      return q.remove();
    } finally { l.unlock(); }}}
```

Seems OK ...

Bounded concurrent queue with two conditions

```
class BoundedBuffer<E> implements BlockingQueue<E> { Lock l; Queue<E> q;
  Condition notFull  = l.newCondition();
  Condition notEmpty = l.newCondition();
  void put(E x) { l.lock(); try {
      while (q.size() == MAX_CAPACITY) { notFull.await(); }
      q.add(x); notEmpty.signal();
    } finally { l.unlock(); }}
  E take() { lock.lock(); try {
      while (q.isEmpty()) { notEmpty.await(); }
      notFull.signal();
      return q.remove();
    } finally { l.unlock(); }}}
```

Seems OK ... Official javadoc:

https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/locks/Condition.html

Concurrent queues

- Bounded array-based buffer

  https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/ArrayBlockingQueue.html

- Optionally limited-length linked list

  https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/LinkedBlockingQueue.html

- Elements with priorities

  https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/PriorityBlockingQueue.html

Lecture plan

1. Decoupling tasks from threads
   - Executor
   - ExecutorService
   - Thread pooling
2. Backoff policies
3. Signalling
4. Condition variable
   - API
   - Lost signal
   - Spurious wakeup
   - Fairness
   - Multiple condition variables per lock
   - Predicate invalidation
5. Thread pools
6. Summary

Predicate invalidation

Short description: thread awaits for predicate and **does not recheck** it upon wakeup

```
threadA() { lock.lock(); try {
        while (true) {
            if (data == null) condition.await();
            System.out.println(data.hashCode())
        }
    } finally { lock.unlock(); }
}
threadB() { while (true) {
        lock.lock(); try {
            if (random.nextBoolean()) { data = null; }
            else { data = new Data(); condition.signal(); }
        } finally { lock.unlock(); }
    }
}
```

Predicate invalidation

Short description: thread `awaits` for predicate and **does not recheck** it upon wakeup
Solution: **always** re-check `await` predicate. If it is invalidated, wait again.
Ensure you are not suffering from missing signal bug.

Condition variable

Conclusion

- Cross-thread message passing/coordination **could** be implemented via mutex + polling
- Signalling saves computation resources for something more useful than busy looping

Mutex + ConditionalVariable allow to build custom synchronization protocols. Be aware of

- Lost signal problem
- Spurious wakeup
- Fairness considerations
- Predicate invalidation

Homework

https://github.com/Svazars/parallel-programming/blob/main/hw/block1/3.5/readme.markdown

Homework

Task 3.5 Open `https://deadlockempire.github.io`*, pass "Condition Variables" level.*

Question time

Question: why are we trying to implement every single bit of `java.util.concurrent`: `Future`, `ExcutorService`, `BlockingQueue`?

Lecture plan

1. Decoupling tasks from threads
   - Executor
   - ExecutorService
   - Thread pooling
2. Backoff policies
3. Signalling
4. Condition variable
   - API
   - Lost signal
   - Spurious wakeup
   - Fairness
   - Multiple condition variables per lock
   - Predicate invalidation
5. Thread pools
6. Summary

ExecutorService: real world API

ExecutorService[14]

```
void execute(Runnable command) // implements Executor
<T> Future<T> submit(Callable<T> task)
<T> T invokeAny(Collection<Callable<T>> tasks) // returns any successful
void shutdown() // no new tasks
List<Runnable> shutdownNow() // try stop all executing, return all pending
boolean awaitTermination(long timeout, TimeUnit unit)
```

Structured concurrency:

- Asynchronous result (`Future`)
- Conditional execution (`invokeAny`)
- Life cycle management (`submit`, `shutdown`, `awaitTermination`)

[14] https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/ExecutorService.html

ScheduledExecutorService

ScheduledExecutorService[15]

```
// Value-returning one-shot task that becomes enabled after the given delay
<V> ScheduledFuture<V> schedule(Callable<V> callable, long delay,
                                TimeUnit unit);
// Periodic action that becomes enabled first after the given initial delay
// Executions will start after `initialDelay`, then `initialDelay + period` ...
ScheduledFuture<?>  scheduleAtFixedRate(Runnable command, long initialDelay,
                                        long period, TimeUnit unit);
```

Delay and periodic schedule control.

Hint: could be useful for testing, stress measurements and simulating "workload bursts".

[15] https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/ScheduledExecutorService.html

ThreadPools API: summary

- `Runnable`, `Callable<T>` – unit of work (e.g. tasks)
- `ThreadFactory` – environment requirements (e.g. threads)
- `Executor` – submission point (e.g. thread-safe container)
- `ExecutorService` – task execution policy (life cycle, resource limits, frequency)

ThreadPools API: summary

- `Runnable`, `Callable<T>` – unit of work (e.g. tasks)
- `ThreadFactory` – environment requirements (e.g. threads)
- `Executor` – submission point (e.g. thread-safe container)
- `ExecutorService` – task execution policy (life cycle, resource limits, frequency)

Bad news: you do not know which thread will execute your code.

- blocking methods, unexpected lock order, deadlocks, race conditions, data races

ThreadPools API: summary

- `Runnable`, `Callable<T>` – unit of work (e.g. tasks)
- `ThreadFactory` – environment requirements (e.g. threads)
- `Executor` – submission point (e.g. thread-safe container)
- `ExecutorService` – task execution policy (life cycle, resource limits, frequency)

Bad news: you do not know which thread will execute your code.

- blocking methods, unexpected lock order, deadlocks, race conditions, data races

Good news: you do not know which thread will execute your code.

- under-the-hood synchronization "just works", nothing to implement

Question time

Question: no deadlock when all submitted tasks have no `Locks` and `Conditions` inside?

ThreadPools API pitfalls: thread-starvation deadlock

```
var service = new SingleThreadExecutorService();
var outerFuture = service.submit(() -> {
    var innerFuture1 = service.submit(() -> {
        return compute(1);
    });
    var innerFuture2 = service.submit(() -> {
        return compute(2);
    });
    return innerFuture1.get() + innerFuture2.get();
});
```

ThreadPools API pitfalls: thread-starvation deadlock

```
var service = new SingleThreadExecutorService();
var outerFuture = service.submit(() -> {
    var innerFuture1 = service.submit(() -> {
        return compute(1);
    });
    var innerFuture2 = service.submit(() -> {
        return compute(2);
    });
    return innerFuture1.get() + innerFuture2.get();
});
```

Single working thread waits for future to be computed by himself.

ThreadPools API pitfalls: resource deadlock

```
var service = new FixedThreadPool(2);
var result = service.submit(() -> {
               return service.submit(() -> {
                 return service.submit(() -> {
                    return compute(1);
                 }).get();
               }).get();;
             }).get();
```

ThreadPools API pitfalls: resource deadlock

```
var service = new FixedThreadPool(2);
var result = service.submit(() -> {
               return service.submit(() -> {
                 return service.submit(() -> {
                    return compute(1);
                 }).get();
               }).get();;
             }).get();
```

`Future.get` that does not block but helps with pending tasks?

ThreadPools API pitfalls: resource deadlock

```
var service = new FixedThreadPool(2);
var result = service.submit(() -> {
               return service.submit(() -> {
                 return service.submit(() -> {
                    return compute(1);
                 }).get();
               }).get();;
             }).get();
```

`Future.get` that does not block but helps with pending tasks?
`CompletableFuture`, `ForkJoinTask` ...

ThreadPools API pitfalls: resource deadlock

```
var service = new FixedThreadPool(2);
var result = service.submit(() -> {
               return service.submit(() -> {
                 return service.submit(() -> {
                    return compute(1);
                 }).get();
               }).get();;
             }).get();
```

`Future.get` that does not block but helps with pending tasks?
`CompletableFuture`, `ForkJoinTask` ... Not today.

Question time

Question: is it always a good idea to use `ExecutorService` that creates threads on demand?

ThreadPools API pitfalls: denial of service

```
var service = new ThreadPerTaskExecutorService();
for (Data request: userRequests) {
    service.submit(() -> {
        return process(request);
    });
}
```

ThreadPools API pitfalls: denial of service

```
var service = new ThreadPerTaskExecutorService();
for (Data request: userRequests) {
    service.submit(() -> {
        return process(request);
    });
}
```

Too many requests could cause `OutOfMemory: too much threads`.

ThreadPools API: available implementations

Executors[16]

```
static ExecutorService newFixedThreadPool(int nThreads, ThreadFactory tFactory)
static ExecutorService newSingleThreadExecutor(ThreadFactory threadFactory);
static ExecutorService newCachedThreadPool(ThreadFactory threadFactory);
```

[16] https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/Executors.html

ThreadPools API: available implementations

Executors[16]

```
static ExecutorService newFixedThreadPool(int nThreads, ThreadFactory tFactory)
static ExecutorService newSingleThreadExecutor(ThreadFactory threadFactory);
static ExecutorService newCachedThreadPool(ThreadFactory threadFactory);
```

Remember:

- Fixed-size thread pools may cause resource deadlock
- Single-thread executor not necessarily use the same thread
- Caching can cause memory leaks or unintentionally reuse resources (e.g. native-lib-specific resources)

Full overview and arbitrary customization available[17]

[16] https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/Executors.html
[17] https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/ThreadPoolExecutor.html

ThreadPools API: now you know it

From now on, you **must** avoid using `new Thread()` in your solutions.

- Use existing `ExecutorServices`
- Create custom `ThreadPoolExecutors`
- Encapsulate thread configuration in `ThreadFactory`

Summary

- Decoupling tasks – `Runnable`, `Callable`
  - from particular threads – `ThreadFactory`
  - from submission protocol – `Executor`
  - from scheduling policy – `ExecutorService`
- Backoff policies for polling – better CPU utilization by sacrificing latency
  - NoBackoff, FixedBackoff, ExponentialBackoff
- Condition variable allows to replace polling with OS-level signalling
  - lost signal, spurious wakeups, fairness, predicate invalidation
- Every thread pool type has
  - advantages: life cycle management, delayed execution, on-demand thread creation
  - weaknesses: thread-starvation deadlock, resource deadlock, denial of service

Summary: homework

Lecture 3, Tasks 3.*: https://github.com/Svazars/parallel-programming/blob/main/hw/block1